

# FROTA FÁCIL RJ

GRUPO L

## **Autores do Projeto:**

Luiz Paulo Corrêa da Silva  
João Vitor Pereira

## 1. Descrição

### **Responsáveis pela seção:**

Luiz Paulo Corrêa da Silva

### **Contextualização**

O Rio de Janeiro (RJ) é a 3ª cidade com pior trânsito no ranking mundial e a primeira no Brasil. Os cariocas perdem, em média, 47% mais tempo nos deslocamentos devido ao fluxo parado do que levariam se os veículos estivessem trafegando na velocidade das vias. Ao final do ano, são 165 horas gastas no trânsito [3].

O trânsito afeta diretamente o transporte por ônibus nas cidades, trazendo o aumento da duração das viagens em horários de pico, momento em que há uma maior demanda de passageiros buscando por deslocamento. Para permitir que essa demanda seja atendida, a Prefeitura do Rio de Janeiro estabelece o intervalo mínimo entre as viagens de uma linha de ônibus ao decorrer do dia. Esse intervalo é conhecido como *headway*.

É dever das empresas de ônibus cumprirem esse intervalo e, caso contrário, estão sujeitas a ações judiciais. Portanto, é essencial que as concessionárias detenham um bom planejamento e programação da operação, disponibilizando o número de veículos necessários entre as linhas de ônibus ao decorrer do dia

Atualmente, a Prefeitura do Rio de Janeiro determina para todas as linhas de ônibus municipais o intervalo mínimo exigido (*headway*) através de diretrizes contratuais, podendo ser acessadas por meio de datasets GTFS (General Transit Feed Specification). No entanto, o dimensionamento correto da frota depende de uma variável altamente imprevisível: o **Tempo**

**de Ciclo** da linha, que é o tempo total gasto para realizar uma viagem redonda (ida e volta), incluindo o tempo parado nos terminais [1].

O dimensionamento da frota é dado pela seguinte equação [1]:

$$F = T \div H_{eadway}$$

Equação 1 - Definição do Dimensionamento da Frota

Sendo F o dimensionamento da frota (número de veículos necessários), T o tempo de ciclo da linha (min) e H o intervalo entre viagens (*headway*)

Já o tempo de ciclo pode ser calculado da seguinte maneira [1]:

$$T = T_{viagem} + T_{Terminais}$$

Equação 2 - Definição do Tempo de Ciclo

Sendo  $T_v$  o tempo total de viagem (ida e volta) e  $T_T$  o tempo que o ônibus ficará parado nos terminais

Suponha que uma linha de ônibus possua Tempo de Ciclo de 60 minutos às 4 da manhã. Para cumprir um headway de 60 minutos, é necessário apenas um ônibus em operação nessa linha[1]. O veículo, ao sair do terminal inicial e completar a rota de ida e volta, estará disponível para percorrer a rota novamente exatamente dentro do intervalo de 60 minutos.

Se às 5 da manhã o tempo de ciclo aumentar para 90 minutos e o headway se mantiver o mesmo, serão necessários pelo menos 2 ônibus em operação. Porém, se às 7 da manhã o tempo de ciclo aumentar para 150 minutos (trânsito) e o headway abaixar para 10 minutos (alta demanda), serão necessários  $150 \div 10 = 15$  ônibus em operação no total da linha, considerando os dois sentidos.

Como o comportamento do trânsito no Rio de Janeiro é extremamente dinâmico ao longo do dia, uma linha que leva poucas horas para completar o ciclo de madrugada pode dobrar de duração no horário de pico, exigindo uma frota muito maior para manter a mesma frequência de atendimento.

Abaixo, podemos observar essa dinamicidade com dados sobre a duração média das viagens e suas velocidades médias dos ônibus municipais do Rio de Janeiro. O gráfico foi gerado com datasets de viagens de ônibus antigas via pandas. Detalharemos a fonte dos dados utilizados em uma seção própria mais a frente.

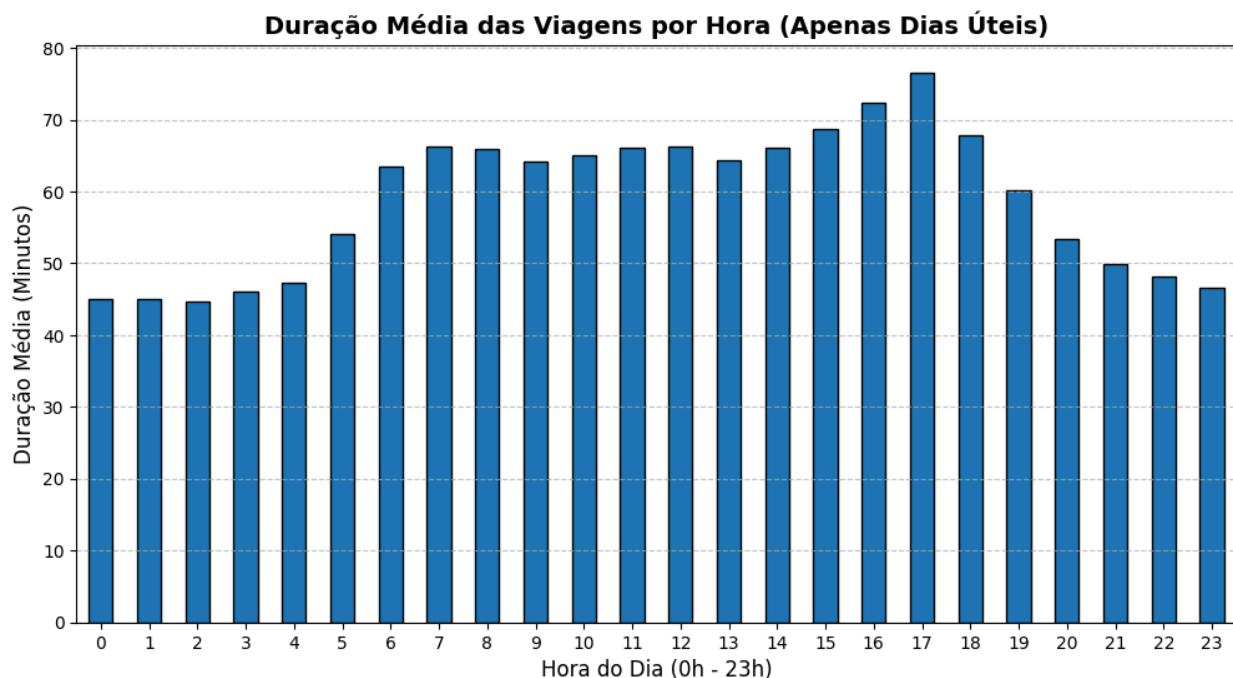


Figura 1 - Duração Média das Viagens Por Hora em Dias Úteis

## Motivações

É possível perceber que o dimensionamento da frota de ônibus deve ser dinâmico ao decorrer do dia, principalmente em um cenário como o do Rio, onde a duração das viagens muda a todo momento por conta do trânsito caótico. Sem ferramentas adequadas para permitir prever o tempo das viagens futuras, as empresas de ônibus podem alocar seus veículos de maneira inadequada e desrespeitar o intervalo entre as viagens (*headway*). Assim, podem estar sujeitas a multas pela prefeitura e gastos desnecessários com combustível, caso aloquem mais veículos para uma linha do que é necessário.

Por outro lado, a população dependente de transportes rodoviários é diretamente afetada. Quando os congestionamentos aumentam, a duração das viagens se expande. Sem sistemas inteligentes para prever a necessidade de frota, as operadoras de transporte não conseguem lidar com a mudança inesperada do tempo de ciclo pelo trânsito, o que aumenta o tempo médio de espera nos pontos e reduz a qualidade do serviço prestado.

Além disso, o dimensionamento dinâmico da frota futura abre espaço para planejamento e remanejamento de veículos entre as linhas de ônibus no decorrer do dia. Se for previsto que em um determinado horário uma linha de ônibus necessite de mais 5 ônibus para respeitar o *headway* previsto, pode-se retirar veículos de linhas em que o dimensionamento da frota diminuiu nesse mesmo horário para atender o aumento da demanda por veículos.

Portanto, fica clara a necessidade de ferramentas capazes de prever o dimensionamento da frota das linhas de ônibus de forma automatizada. Assim, propomos

neste relatório uma ferramenta de Apoio à Tomada de Decisão voltada para empresas de ônibus e que seja capaz de prever o tempo de ciclo de viagens futuras e automatizar o cálculo da frota necessária por linha e por dia.

## **Limitações e Escopo**

O dimensionamento da frota não fornece o itinerário dos ônibus ou o espaçamento necessário entre eles para garantir que o headway seja atingido. Tais métricas são obtidas com outros dados dentro do processo de Planejamento de Operação e não serão abordadas neste relatório.

Futuramente, tais cálculos poderiam ser abordados e, com base no dimensionamento da frota, gerar automaticamente o itinerário da linha com os horários de saída e chegada. Conforme [1], para realizar isso, são necessárias métricas de número de passageiros e da detecção de trechos que trazem gargalos na duração da viagem.

As métricas acima poderiam ser obtidas por meio da análise do GPS das linhas de ônibus em tempo real, bem como dos dados da bilhetagem de cada veículo. Além disso, acreditamos que em um futuro momento os dados das rotas de ônibus em combinação com dados de GPS poderiam ser utilizados para melhorar a precisão do modelo de Machine Learning utilizado. Ao invés de olhar apenas para dados de linhas de ônibus de forma separada, poderíamos prever a duração da viagem em trechos comuns entre ônibus de linhas diferentes.

Entretanto, optamos por realizar uma análise macroscópica, focando apenas nas durações totais das viagens, sem utilizar dados gerados durante o andamento de uma viagem. Note que, segundo [1], "O estudo do movimento de veículos de transporte público em nível microscópico é importante nos estudos teóricos. No entanto, para a elaboração da programação operacional é suficiente o modelo macroscópico do movimento entre os pontos terminais das viagens (onde elas têm início e fim)".

Por fim, outra limitação inerente ao modelo de Inteligência Artificial atual é a sua cobertura das linhas de ônibus. Para garantir a confiabilidade das previsões, linhas de ônibus com histórico de dados escassos foram removidas do escopo. Mais à frente detalharemos a fundo o critério da exclusão e o impacto no dataset original

# **2. Objetivos**

## **Responsáveis pela seção:**

Luiz Paulo Corrêa da Silva

## **2.1 Objetivo Geral**

Desenvolver um Sistema de Apoio à Tomada de Decisão fundamentado em Inteligência Artificial para prever o tempo de ciclo de linhas de ônibus e automatizar o cálculo do dimensionamento dinâmico da frota, garantindo o cumprimento das diretrizes contratuais de headway no município do Rio de Janeiro e abrindo portas para futuras otimizações via remanejamento da frota entre linhas de ônibus.

## 2.2 Objetivos Específicos

1. Definir quais tabelas e colunas do dataset serão utilizadas para o treinamento do modelo
2. Coletar e analisar tabelas que serão utilizadas para identificar outliers e realizar a limpeza.
3. Processar tabelas para tratar valores nulos, duplicados ou inconsistentes.
4. Cruzar dados de tabelas via joins para criar duas tabelas finais em formato csv: Uma para treinamento do modelo e outra com informações do headway de cada linha, contendo a hora e dia da semana no qual cada headway é válido
5. Converter tipos de valores da tabela final para maximizar a eficiência do treinamento do modelo
6. Identificar qual modelo é mais adequado para prever a duração das viagens
7. Desenvolver algoritmo para treinamento e validação do modelo
8. Desenvolver algoritmo que implemente metodologia de cálculo de previsão do dimensionamento da fórmula com base em [1], via modelo de ML criado anteriormente.
9. Identificar headway válido para o momento da previsão via filtragem de tabela.
10. Gerar visualizações gráficas que permitam às concessionárias identificar picos de demanda por frota e otimizar a alocação de recursos ao longo da operação.

Abaixo, é possível ver um diagrama que demonstra o mapeamento dos objetivos propostos. Note que os objetivos foram divididos em fases. Cada fase foi implementada em um arquivo .ipynb específico.

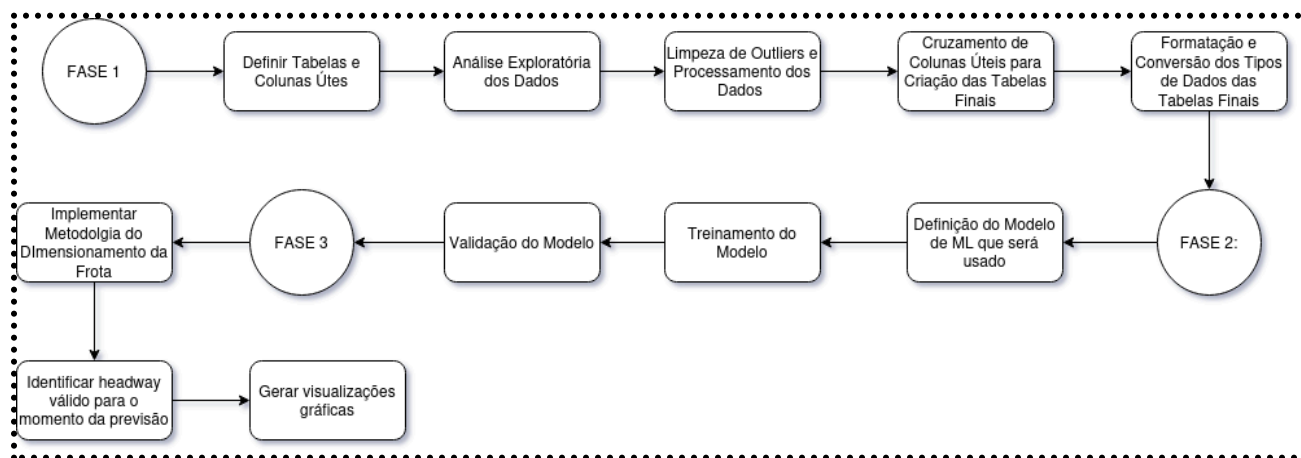


Figura 2 - Diagrama com Objetivos e Etapas do Projeto Proposto

# 3. Dados Utilizados

## Responsáveis pela seção:

Luiz Paulo Corrêa da Silva

Para o desenvolvimento do modelo preditivo de duração das viagens e o consequente cálculo de dimensionamento de frota, foi utilizado a coletânea de datasets **SMTR** — Sistema Municipal de Transportes disponibilizado pela equipe da UFRJ Analytica.

Nessa coletânea, diversos datasets sobre transporte público estavam disponíveis. Focamos nos transportes por ônibus e selecionamos dois principais. Abaixo, iremos abordá-los.

## GTFS (General Transit Feed Specification) [4]

É um formato padronizado mundialmente para dados de transporte público, criado pelo Google e agências de trânsito. Ele permite que agências publiquem horários, rotas e paradas em arquivos.

O GTFS do Rio possui diversas tabelas de dados diferentes. Abaixo, temos uma descrição dos dados presentes, bem como a quantidade inicial de entradas disponíveis para análise:

1. Paradas de ônibus e suas localizações (485884 linhas)
2. Geometria das rotas de ônibus (13946 linhas)
3. Calendário dos headways definidos da operação regular e eventualidades como obras (34225 linhas)
4. Horário e dia da semana de funcionamento das linhas de ônibus com tamanho do trajeto percorrido (113629 linhas).

No escopo da nossa solução, as informações de número 3 e 4, expostas acima, serão necessárias. Esses dados estão distribuídos em 3 tabelas diferentes. Abaixo, detalharemos as tabelas com as colunas úteis para a nossa solução

### **frequencies.csv (484555 linhas)**

1. start\_time e end\_time. String com dados sobre hora do dia que a regra do headway deve valer.
2. headway\_secs . Inteiro 64 bits armazenando o tempo de headway definido.

3. trip\_id. Chave estrangeira do tipo String para tabela de trips, contendo a linha de ônibus afetada pela regra

### **trips.csv (556360 linhas)**

1. trip\_id. Utilizado para join com a tabela acima
2. trip\_short\_name: Nome da linha (ex: 324). Tipo String
3. direction\_id: Direção ida ou volta. Representado por número binário sendo 0 a ida e 1 a volta.
4. service\_id: Chave estrangeira do tipo String para tabela de calendar.csv.

### **calendar.csv (923 linhas)**

1. service\_id: Utilizado para dar join com a tabela acima
2. Coluna dias da semana: 7 colunas binárias informando os dias da semana onde a operação é válida.
3. start\_time e end\_time: Intervalo em data (dd/mm/aaaa) no formato de String contendo o período em que a operação está em vigor.

### **ordem\_servico\_csv (113629 linhas)**

1. servico: String contendo o nome da linha
2. extensao\_ida e extensao\_volta: float contendo a extensão do trajeto de ida e de volta da linha.

O objetivo é cruzar as tabelas frequencias, trips e calendar, para podermos ter todos os dados de headways por linha, data, dia da semana e sentido. Mais à frente detalharemos esse processo.

## **br\_rj\_riodejaneiro\_viagem\_zirix (256725 linhas)**

Dataset contendo uma tabela (viagem\_informada.csv) com diversos dados de viagens de antigas entre os pontos terminais. Abaixo temos a lista de colunas que serão utilizadas:

1. servico: String que será usada para cruzar com a tabela servico da GTFS e identificar a linha de ônibus.
2. sentido: Direção ida ou volta. Representado por número binário sendo 0 a ida e 1 a volta.
3. datetime\_partida e datetime\_chegada: Usados para calcular o tempo de duração de uma viagem de ônibus

Para essa tabela, criaremos a variável duracao\_viagem, por meio da operação datetime\_chegada - datetime\_partida. Esse será o dado que o modelo deve advinhar.

O objetivo é cruzar essa tabela com `ordem_servico` da GTFS para que se possa calcular a velocidade média da viagem, critério que será utilizado para identificar outliers. Mais à frente detalharemos esse processo.

## Pré-processamento

Inicialmente, testamos se haviam valores nulos nas tabelas `frequencies.csv`, `trips.csv` e `viagem_informada.csv`. Nessa busca, encontramos apenas 9 itinerários de linhas sem direção de sentido definida. Percebemos que eram itinerários excepcionais, como operações para reveillon ou alternativas. Decidimos excluí-las.

Em seguida, verificamos a distribuição dos dados da coluna `headway_secs` da tabela `frequencies.csv` via Boxplots. Abaixo, pode-se encontrar a figura gerada.

Temos poucos valores distantes próximos a 10.000 segundos. Isso equivale a aproximadamente 3 horas. Provavelmente são ônibus que funcionam de madrugada ou que possuem poucos horários por dia. Além disso, não temos tempo de headway muito pequenos. Assim, não temos outliers que necessitem ser excluídos.

Em seguida, adicionamos a coluna `duracao_viagem` na tabela `viagens_informadas`, realizando um delta entre o tempo final - inicial. Para isso, utilizamos bibliotecas do `datetime` do `pandas` que leram automaticamente a string de data e hora, permitindo a extração rápida da hora, minutos e segundos após meia noite.

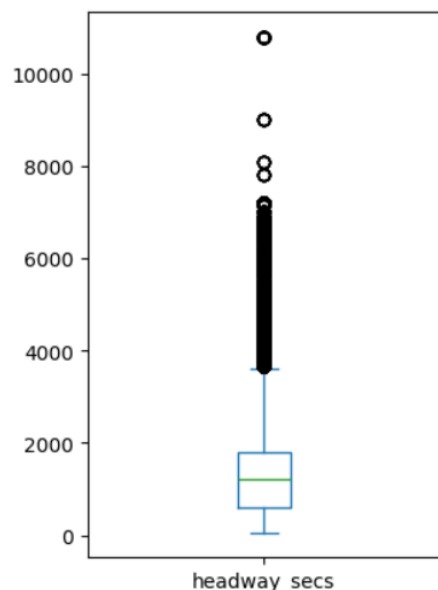


Figura 3 - Boxplot da Coluna `headway_secs`

## Limpeza de Linhas com Poucas Viagens

Na tabela `viagem_informada.csv`, uma das preocupações sobre o dataset era se o número de viagens disponível por linha seguia uma distribuição uniforme. Sabemos que, caso contrário, o modelo teria dificuldade para prever a duração de viagens para linhas com poucas entradas disponíveis. Assim, torna-se necessário verificar como esses dados estão distribuídos.

Nesse sentido, implementamos um algoritmo via `pandas` que contava quantas viagens possuíam o mesmo valor para a linha de ônibus e fizemos um histograma. Iremos perceber que esses dados estão mal distribuídos. Com a ajuda de Inteligência Artificial, adicionamos uma linha vertical no gráfico, marcando a fronteira entre o número de viagens com mais ou menos 200 viagens.



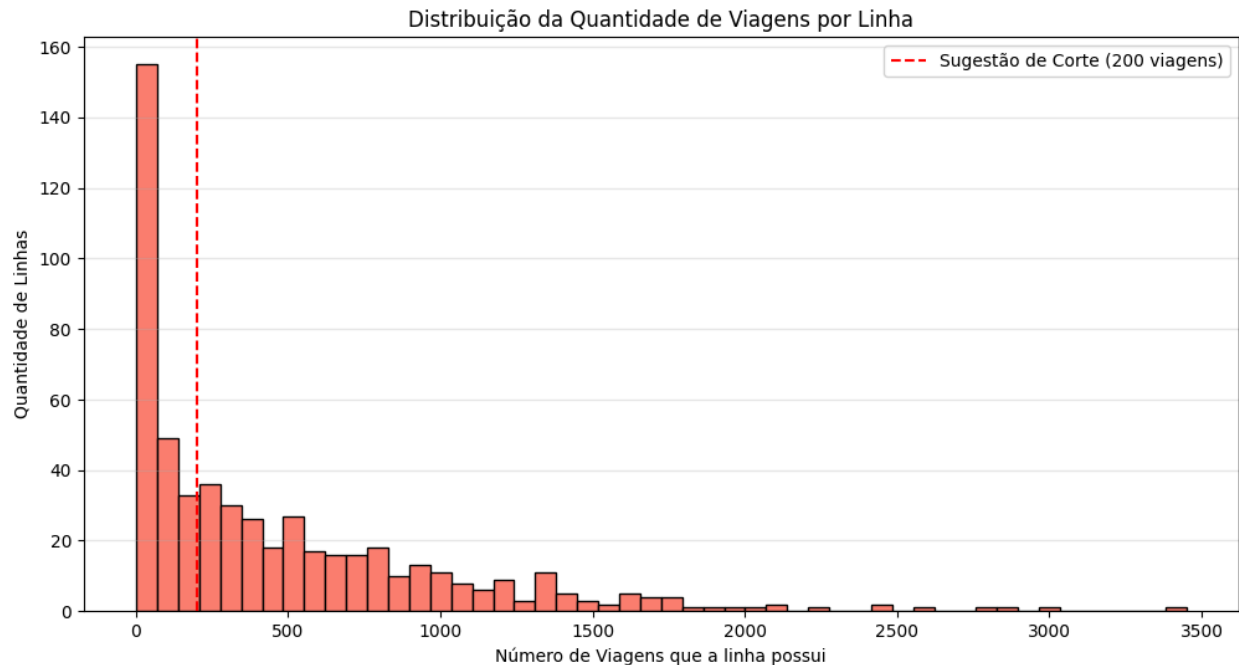


Figura 4 - Distribuição da Quantidade de Viagens Por Linha

Além do gráfico acima, também geramos as seguintes saídas:

Total de linhas: 549

Linhas com menos de 200 viagens: 235

Linhas com mais ou igual de 200 viagens: 314

Total de viagens: 256725 viagens

Quantidade de viagens de linhas pequenas: 12923 viagens

Porcentagem equivalente do total de dados: 5.03%

Portanto, para evitar que o modelo fosse alimentado com parâmetros mal distribuídos, decidimos excluir todas as viagens à esquerda da linha vermelha. Perdemos a capacidade de prever 235 de 549 linhas, porém, das linhas restantes, temos uma quantidade segura de entradas. Além disso, o impacto no dataset foi baixo, representando a exclusão de apenas 5% das viagens.

### Limpeza de viagens excessivamente curtas ou longas

Ainda na tabela viagens\_informadas.csv, decidimos verificar como os valores da coluna duração estavam distribuídos. Havia a presença de viagens excessivamente curtas (ex. 2 minutos) ou excessivamente longas (ex. 8 horas) na tabela? Nesse caso, sabemos que

se trata de outliers e erros de medição, pois, não é possível que um ônibus faça o trajeto de ida ou volta em um tempo tão curto.

Utilizamos a função `pandas describe()` para visualizar os valores mínimos e máximos dessa coluna, encontrando, respectivamente, -0,5 e 728,12 minutos de duração da viagem. Além disso, geramos um histograma dessa coluna, aplicando zoom em 3 faixas diferentes do gráfico. Segue a visualização abaixo:

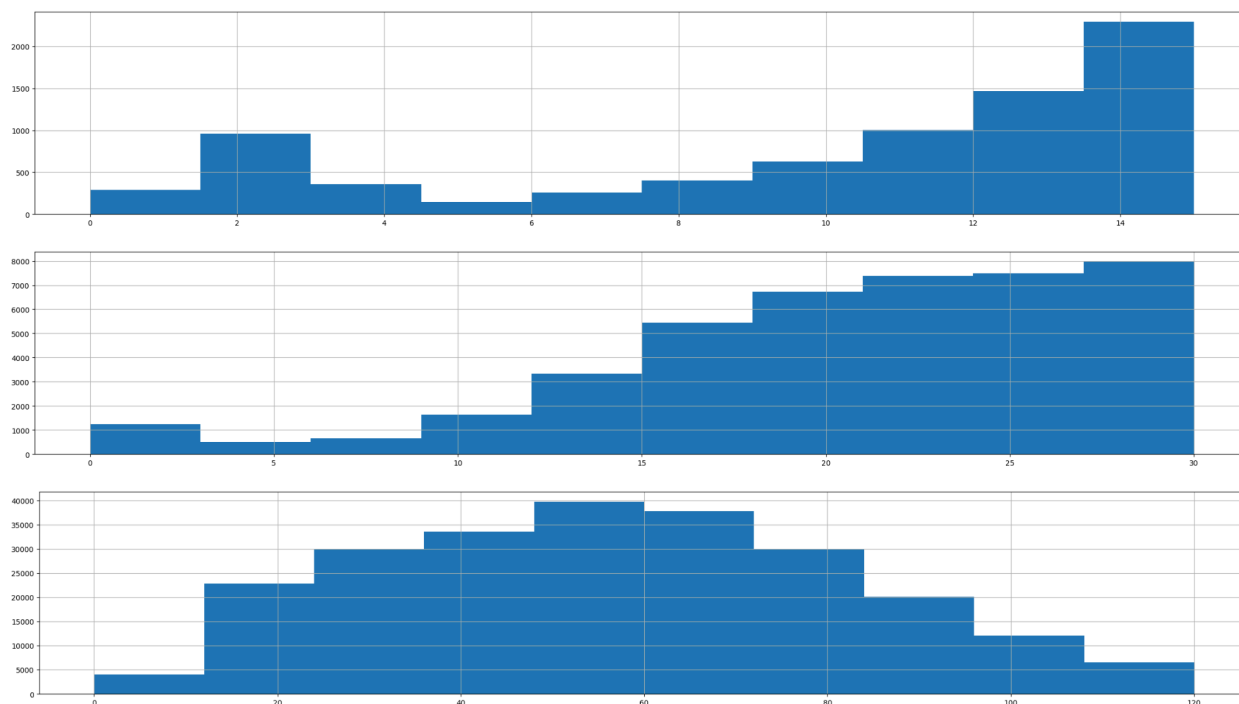


Figura 5 - Histogramas dos dados de duração da viagem

É possível perceber a presença de muitos outliers, pois, há uma quantidade considerável de viagens com duração entre 0 e 10 minutos. Para identificá-los, desenvolvemos a função `outlier`, presente no COLAB. Uma função semelhante foi apresentada na aula de Tratamento de Dados da Analytica, mas, optei por essa pois os dados de trânsito não são simétricos. Utilizar apenas IQR não foi eficiente.

Inicialmente, excluimos todas as viagens mais curtas que 10 minutos e tentamos filtrar usando intervalos interquartis e medianas, porém, isso acabou permitindo viagens muito rápidas/curtas ou então excluindo viagens rápidas.

Percebemos que apenas o tempo de viagem não era uma boa medida para identificar essas viagens, pois, sabemos que o tempo gasto em um percurso é diretamente proporcional ao tamanho do trajeto. Portanto, como melhoria, cruzamos as tabelas de `viagem_informadas` com `ordem_servico` (GTFS). Assim, foi possível obter a distância percorrida para cada viagem.

Em seguida, adicionei a coluna velocidade média, dividindo a distância percorrida pela duração da viagem. Agora, podemos realizar a filtragem com base na variável velocidade

média, trazendo maior precisão na nossa limpeza. Segundo [5], a velocidade máxima permitida por ônibus municipais é de 60 km/h, com exceção de vias específicas. Assim, decidimos, excluimos todas as linhas com velocidade média fora do intervalo [5, 65] km/h.

Em seguida, após realizar a limpeza desses absurdos, agrupamos as viagens por linha de ônibus e aplicamos técnicas de limpeza estatística via intervalos interquartis e valor da mediana desses grupos. Considerando a variável de velocidade média.

Tivemos o seguinte resultado:

Viagens com velocidade nula - 8454

Viagens com velocidade absurda (fora da faixa 5 a 65 km/h): 12617

Limpamos as viagens acima e, em seguida, realizamos a limpeza estatística.

## **Formatação Final da Tabela: Convertendo os Valores Para o Modelo de ML**

Para o modelo preditivo, decidimos utilizar um modelo baseado em árvores. Mais especificamente, utilizamos o RandomForestRegressor da biblioteca Scikit-learn da linguagem Python. Detalharemos os motivos da escolha em uma seção própria mais abaixo.

Segundo [2], "when using non-linear models such as decision trees (or ensembles of thereof), we do not explicitly encode features such as month number or day of the year as dummies. Those models are capable of learning non-monotonic relationships between ordinal input features and the target". Logo, não precisaremos utilizar técnicas de One Hot Encoding e Dummy Variables no treinamento do modelo.

Assim, as colunas que alimentarão o modelo serão:

1. mes\_viagem: valor inteiro de 1 a 12
1. dia\_viagem: valor numerico (1 a 30) do dia da viagem.
2. dia\_semana: valor inteiro de 1 a 7 (segunda a domingo)
1. eh\_feriado: 0 se for feriado e 1 se nao. Definido utilizando a biblioteca holidays da linguagem Python.
2. hora\_partida: valor inteiro representando a quantidade de minutos que se passaram desde meia noite (0 até 1429)
2. duracao: valor inteiro representando a quantidade de minutos que se passaram desde meia noite (0 até 1429). **Essa será a previsão do modelo.**
3. linha: numero representando a linha. sera necessário fazer um hashmap entre a string da linha (ex. "SN324") e um numero inteiro representando-a
3. sentido: Indica se a viagem estava indo (0) ou voltando (1)

Implementamos o algoritmo que calcula essas novas colunas, com base nas colunas já existentes na tabela resultante da junção de viagem\_informada e ordem\_servico, gerando a tabela final pronta para treinar o modelo. Note que, segundo [6], “no normalization is required for RandomForestRegressor”. Portanto, não normalizamos os dados antes do treinamento.

Além disso, decidimos não considerar o ano da viagem para treinar o modelo. A dinâmica do trânsito no Rio de Janeiro é governada por ciclos que se repetem (horários de pico ao longo do dia, dias úteis x fins de semana, férias escolares nos meses de janeiro e julho, e feriados). A passagem linear dos anos não é uma variável causal direta para o tempo de viagem.

### Cruzamento de trips, frequências e calendar

Além da tabela final utilizada para treinar o modelo de ML, precisamos cruzar dados de três tabelas diferentes, trips, frequencies e calendar. Isso será necessário para consultar os dados do headway de cada previsão.

Uma outra dificuldade é que o intervalo da duração da regra do GTFS está em um formato que permite mais de 24 horas, sendo o horário depois de 24 equivalente à madrugada do dia seguinte. Precisamos primeiro converter a hora para minutos depois de meia noite, seguindo o mesmo padrão da tabela que será usada pelo modelo.

Implementamos um algoritmo via pandas que realiza o cruzamento via joins e converte os valores de data. Além disso, também foi necessária a exclusão de duplicatas e colunas inúteis durante o processo.

## 4. Métodos

### Responsáveis pela seção:

Luiz Paulo Corrêa da Silva

### 4.1 Pipeline

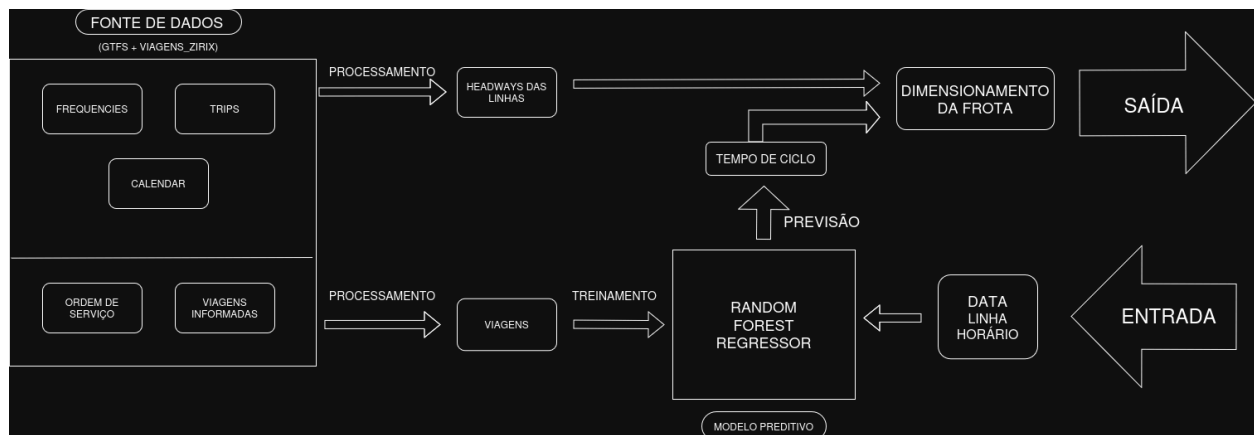


Figura 6 - Pipeline dos Dados

## 4.2 Técnicas e Algoritmos

Para escolher o tipo de modelo que seria utilizado, pesquisamos por quais modelos de ML são recomendados para realizar análise temporal em um contexto de dados não lineares. Nesse sentido, encontramos dois tipos principais: Regressão e Modelos Baseados em Árvores.

Inicialmente, pensamos em implementar uma regressão polinomial que prevê a duração das viagens ao longo do dia. Porém, isso se mostrou inviável, pois, precisaríamos gerar um modelo diferente para todas as linhas e um modelo específico para cada dia da semana dentro de uma linha, incluindo o caso de ser feriado ou não.

Portanto, decidimos seguir com os modelos baseados em árvores. Nesse sentido, escolhemos o RandomForestRegressor, implementado na biblioteca Scikit-learn do Python. Ao inserir os parâmetros da viagem, como o sentido, dia, mês, hora e código da linha, o RandomForest é capaz de mapear automaticamente o comportamento individual de cada linha, utilizando diversas ramificações de árvores de decisão.

Além de resolver o problema da não-linearidade do trânsito, a escolha deste algoritmo otimizou profundamente o desenvolvimento do projeto. Como fundamentado na seção de pré-processamento, o Random Forest lida de forma nativa com variáveis ordinais (como os inteiros de tempo extraídos) sem a necessidade de *One-Hot Encoding*, e é insensível à escala das variáveis, o que dispensou etapas custosas de normalização [2, 6].

Assim, separamos 80% do dataset para treino e 20% para validação, seguindo o tutorial [7]. Para validar nosso modelo, calculamos as métricas de Erro Médio Absoluto, Erro Médio Quadrático e  $r^2$  Score. Abaixo, segue uma tabela contendo os valores encontrados.

| Métrica                             | Valor Obtido |
|-------------------------------------|--------------|
| MAE (Erro Absoluto Médio)           | 7.06         |
| MSE (Erro Quadrático Médio)         | 111.82       |
| $R^2$ (Coeficiente de Determinação) | 0.8617       |

Figura 7 -Tabela com as Métricas para Validação do Modelo de ML

Note que os resultados são satisfatórios. Um erro absoluto médio de 7 minutos, quando a média de duração das viagens é de 60 minutos não compromete o dimensionamento da frota. Além disso, o coeficiente de determinação é de 86%.

Embora o Erro Quadrático Médio (MSE) apresente um valor elevado (111.82), é importante ressaltar que sua métrica é dada ao quadrado ( $\text{minutos}^2$ ). Ao calcularmos a raiz deste valor, obtemos um desvio de aproximadamente 10,5 minutos. Isso comprova que, mesmo quando a métrica penaliza os maiores desvios da predição, a margem de erro se mantém restrita e viabiliza a solução proposta.

Por fim, vale ressaltar que tivemos que utilizar um Label Encoder para codificar numericamente o número das linhas de ônibus. A conversão de String para Inteiro não seria possível pois existem linhas de ônibus que possuem caracteres em seu nome (ex. SN324).

## 4.3 Ferramentas e Tecnologias

Para o desenvolvimento da solução, desde a etapa de coleta e processamento de dados até o treinamento e avaliação do modelo de *Machine Learning*, foram utilizadas diversas bibliotecas da linguagem Python. Abaixo, detalhamos o papel de cada uma no projeto:

- **Pandas:** Foi a principal ferramenta para manipulação e análise de dados
- **NumPy :** Foi empregada para a realização de operações matemáticas e manipulação de matrizes durante os cálculos de distância e velocidade.
- **Scikit-Learn:** A biblioteca para a modelagem preditiva do projeto. A partir dela, implementamos o RandomForestRegressor, treinamento e validação do modelo, métricas de validação do modelo e Label Encoder para linhas de ônibus.
- **Matplotlib e Seaborn:** Bibliotecas dedicadas à visualização de dados, permitindo a geração de gráficos.
- **Holidays:** Biblioteca utilizada para identificar automaticamente se a data de uma viagem caía em um feriado.
- **Joblib:** Ferramenta utilizada para exportar e salvar o modelo de *Random Forest* após o treinamento, permitindo acessar de outro colab
- **Datetime e Math:** Bibliotecas nativas do Python. A datetime foi utilizada para o tratamento preciso de atributos temporais (meses, dias e horas extras), e a math serviu como apoio em cálculos numéricos.

Todos os algoritmos foram implementados em ambiente de nuvem via COLAB. Cada fase do projeto foi implementada em um colab específico e pode ser acessada abaixo:

1 -  Coleta EDA e Processamento de\_dados PS.2026 Grupo L

2 -  Treinamento do Modelo PS.2026 Grupo L

3 -  Visualização dos dados PS.2026 Grupo L

## 5. Conclusão

## Responsáveis pela seção:

Luiz Paulo Corrêa da Silva

Com o modelo treinado e os dados de headway limpos, implementamos a metodologia do dimensionamento da frota com base em [1], definida anteriormente nas equações 1 e 2. Abaixo, detalharemos o pseudocódigo do algoritmo implementado para cálculo do tempo de ciclo.

Sendo:

- $T$  = Tempo de Ciclo
- $T_{AB}$  = tempo de percurso do terminal A até o terminal B
- $T_{BA}$  = tempo de percurso do terminal B até o terminal A
- $T_A$  = Tempo parado no terminal A
- $T_B$  = Tempo parado no terminal B

Temos o seguinte pseudocódigo para prever o tempo de ciclo:

```
Entrada (t(hora da viagem), linha, data, eh_feriado, etc)

1. t_inicial = t

2. Calculamos a previsão ( $T_{ab}$ ) via modelo preditivo para duração da
viagem de ida dado o tempo t e demais informações

3. t +=  $T_{ab}$ 

4. t +=  $T_b$  (10min)

5. Calculamos a previsão ( $T_{ba}$ ) via modelo preditivo para duração da
viagem de volta dado o tempo t e demais informações

6. t +=  $T_{ba}$ 

7. t +=  $T_a$  (10min)

8. tempo_ciclo = t

Retorne tempo_ciclo
```

Figura 8 - Pseudocódigo da função que prevê o tempo de ciclo

Com o tempo de ciclo previsto para um determinado momento no futuro, resta obter qual é o headway daquele momento. Para isso, implementamos uma função que recebe a

tabela completa de headways e filtra a regra válida mais recente para aquela viagem, via funções com pandas. Assim, é possível calcular o dimensionamento da frota para um instante no tempo.

Por fim, implementamos uma função que, dado uma linha de ônibus e uma data futura, calcula o dimensionamento da frota de 10 em 10 minutos e gera um gráfico com os valores encontrados. Segue abaixo a visualização.

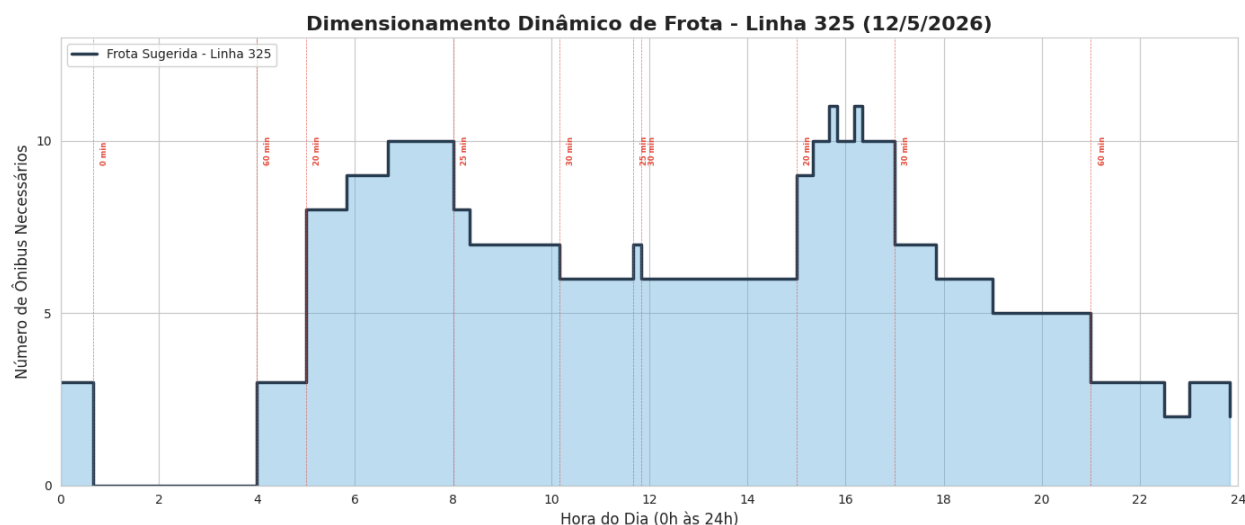


Figura 9 - Dimensionamento Dinâmico da Frota da linha 325 em 12/5/2026

Note que as linhas vermelhas indicam os momentos em que uma nova regra para headway entrou em vigor.

## Limitações

Algumas limitações devem ser discutidas nessa seção. Primeiramente, por mais que as métricas do modelo tenham sido satisfatórias, é importante que futuramente seja analisado o erro do modelo por linha de ônibus. Em seguida, seria interessante procurar correlações entre o erro do modelo por linha e a quantidade de dados de viagens disponíveis para essa linha.

Outra limitação está presente na implementação do cálculo do tempo de ciclo. Tivemos que considerar que os ônibus gastam sempre 10 minutos nos terminais iniciais e finais após terminar uma viagem de ida/volta. Sabemos que esse valor arbitrário não corresponde à realidade. Em uma evolução dessa solução, é importante buscar outras maneiras de calcular esse tempo.

Por fim, é possível observar incoerências em alguns gráficos de dimensionamento da rota. Alguns headways estão em vigor por um período muito curto de tempo, causando mudanças bruscas no dimensionamento da frota de ônibus. É necessário revisar os dados da tabela e o algoritmo que filtra a regra do headway válida, para que se possa descobrir a causa do problema. No gráfico abaixo, esse efeito pode ser observado mais intensamente.



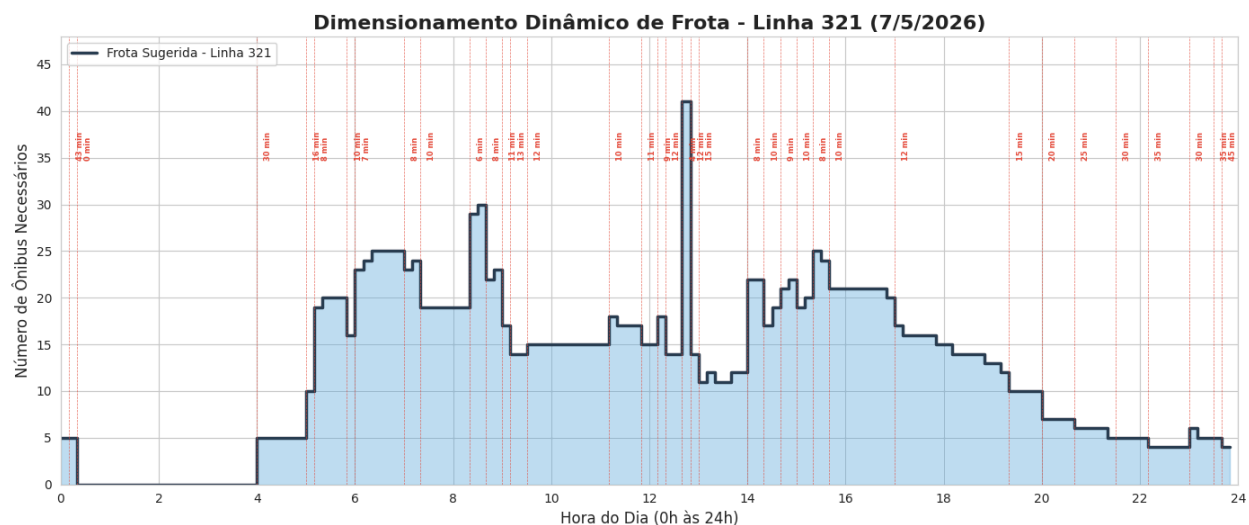


Figura 10 - Dimensionamento Dinâmico da Frota da linha 321 em 7/5/2026

## 6. Referências

- [1] BEZERRA, B. S. **Planejamento e Programação da Operação**: Dimensionamento de Frota. Bauru: Universidade Estadual Paulista (UNESP), Departamento de Engenharia Civil, 2013. Disponível em: [https://pessoas.feb.unesp.br/barbara/files/2013/08/DIMENSIONAMENTO\\_FROTA.pdf](https://pessoas.feb.unesp.br/barbara/files/2013/08/DIMENSIONAMENTO_FROTA.pdf). Acesso em: 7 maio 2026.
- [2] LEWINSON, E. Three Approaches to Encoding Time Information as Features for ML Models. **NVIDIA Technical Blog**, 17 fev. 2022. Disponível em: <https://developer.nvidia.com/blog/three-approaches-to-encoding-time-information-as-features-for-ml-models/>. Acesso em: 7 maio 2026.
- [3] AGÊNCIA CNT TRANSPORTE ATUAL. **Brasil tem cinco cidades entre as mais congestionadas do mundo**. Brasília: Confederação Nacional do Transporte (CNT), 28 set. 2016. Disponível em: <https://cnt.org.br/agencia-cnt/brasil-tem-cinco-das-cidades-mais-congestionadas-do-mundo>. Acesso em: 7 maio 2026.

- [4] ESPECIFICAÇÃO GERAL DE FEED DE TRÂNSITO. In: WIKIPÉDIA, a enciclopédia livre. Flórida: Wikimedia Foundation, 2023. Disponível em: <[https://pt.wikipedia.org/w/index.php?title=Especifica%C3%A7%C3%A3o\\_Geral\\_de\\_Feed\\_de\\_Tr%C3%A2nsito&oldid=66852657](https://pt.wikipedia.org/w/index.php?title=Especifica%C3%A7%C3%A3o_Geral_de_Feed_de_Tr%C3%A2nsito&oldid=66852657)>. Acesso em: 25 out. 2023.
- [5] Prefeitura determina velocidade máxima de 50 km/h para ônibus municipais. Acesso em: <https://www.rio.rj.gov.br/web/guest/exibeconteudo?id=1893686>
- [6] KUNDAN. **Random Forest Regressor**. Kundan's Read, 2024. Disponível em: [https://kundan-reads.readthedocs.io/en/latest/aiml/machine\\_learning/random\\_forest\\_regressor/](https://kundan-reads.readthedocs.io/en/latest/aiml/machine_learning/random_forest_regressor/). Acesso em: 8 maio 2026.
- [7] RYAN & MATT DATA SCIENCE. **Random Forest Regressor in Python: A Step-by-Step Guide**. YouTube, 24 out. 2023. Disponível em: <https://www.youtube.com/watch?v=YUsx5ZNIYWc>. Acesso em: 5 maio 2026.